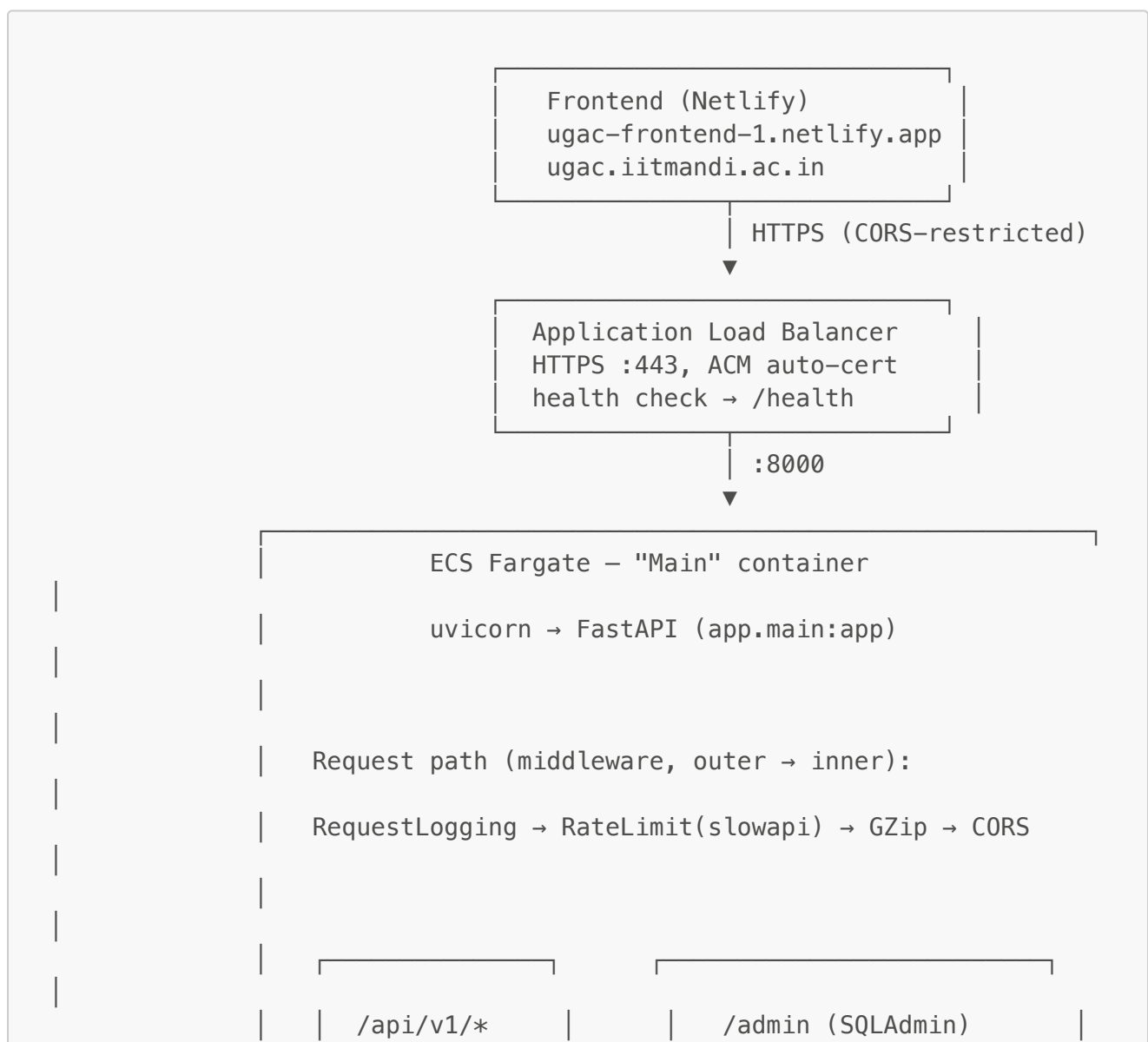


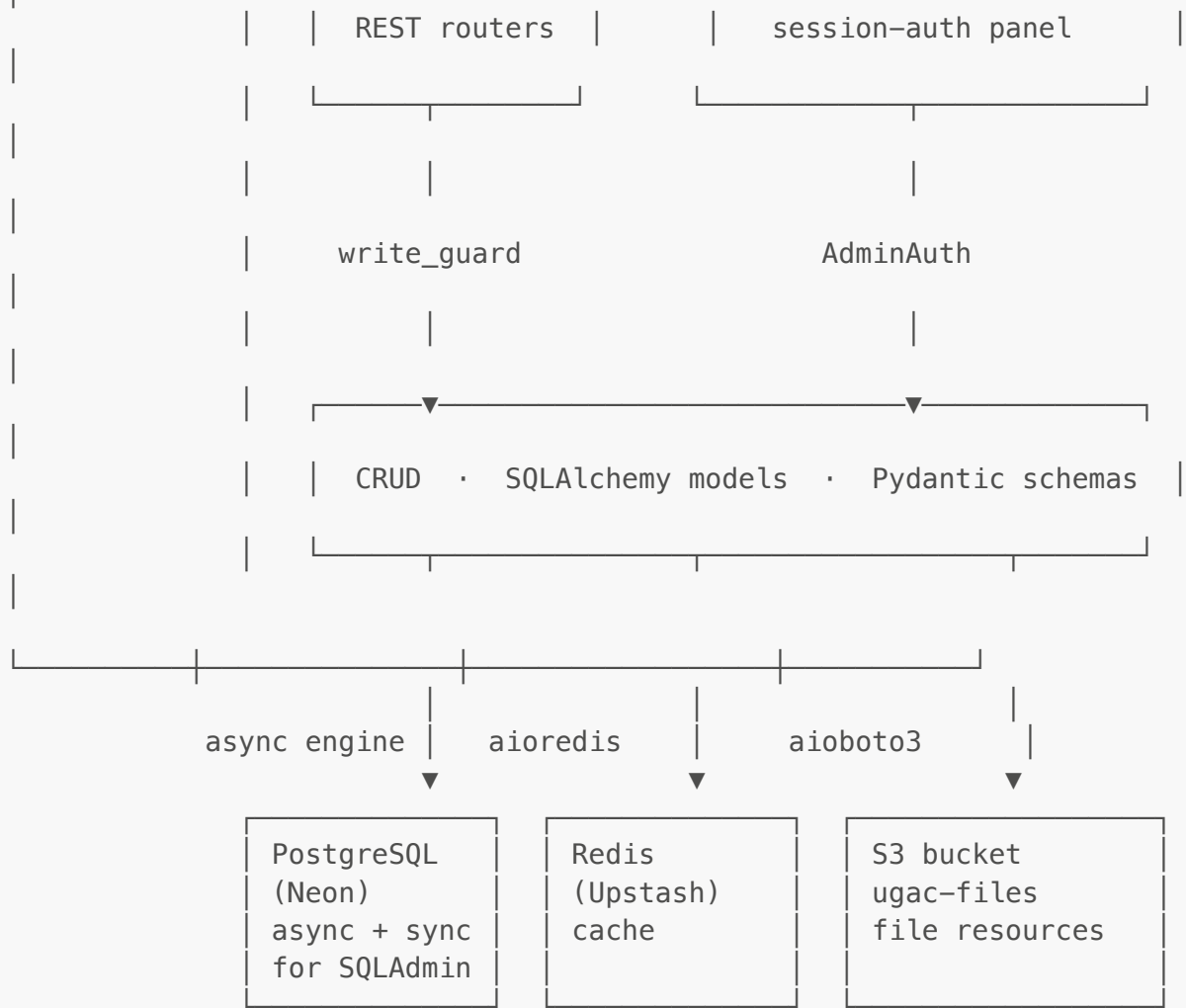
UGAC Web — UG Academic Council, IIT Mandi

Backend API for the Undergraduate Academic Council website. Serves course catalogs, curricula, faculty, reviews, announcements/events, team info, and file resources, with a built-in admin panel for content management.

- **Framework:** FastAPI (async)
- **Database:** PostgreSQL (Neon) via async SQLAlchemy 2.0 + asyncpg
- **Cache:** Redis (Upstash)
- **Storage:** AWS S3 (prod) / MinIO (dev) / local FS — pluggable
- **Admin:** SQLAdmin panel at </admin>
- **Docs:** OpenAPI/Swagger at </docs>
- **Deploy:** AWS ECS Fargate (Express Mode) behind an auto-provisioned ALB; CI/CD via GitHub Actions

Architecture





Secrets: AWS Secrets Manager (prefix `ugac/`) → injected as container env vars
Migrations: Alembic `upgrade head` runs on container start (start.sh)

Layers (backend/app/)

Layer	Path	Role
Entrypoint	<code>main.py</code>	App bootstrap, middleware, admin mount, S3 bucket init
Config	<code>config.py</code>	Env-driven settings, rejects insecure defaults
API routers	<code>api/v1/*</code>	REST endpoints, grouped per resource
Admin views	<code>admin/*</code>	SQLAdmin CRUD UI (12 models)
Core	<code>core/*</code>	auth, deps, cache, storage abstraction, middleware
CRUD	<code>crud/*</code>	DB query logic
Models	<code>models/*</code>	SQLAlchemy ORM tables
Schemas	<code>schemas/*</code>	Pydantic request/response shapes

Layer	Path	Role
DB	<code>database.py</code>	Async engine + session, Redis client factory

API surface (`/api/v1`)

`branches` · `departments` · `courses` · `curricula` · `search` · `reviews` · `team` · `faculty` · `announcements` · `events` · `resources` · `storage`

Read endpoints are public; write endpoints are gated by `write_guard`. The `/admin` panel uses session-based `AdminAuth`. Note: SQLAlchemy needs a **synchronous** engine, so `main.py` derives a sync URL (`+asynccpg` stripped, `ssl=>sslmode=`) alongside the async one.

Local development

Prerequisites: Python 3.12, Docker.

```
cd backend

# 1. Start Postgres + MinIO
docker compose up -d

# 2. Set up env
cp .env.example .env
# For local dev, set in .env:
#   DATABASE_URL=postgresql+asynccpg://ugac:ugac_dev@localhost:5432/ugac
#   STORAGE_ENDPOINT=http://localhost:9000 (MinIO) or local (local FS)
#   SECRET_KEY=<32+ random chars> (required – defaults are rejected)
#   ADMIN_PASSWORD=<something> (required)

# 3. Install deps
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 4. Run migrations
alembic upgrade head

# 5. Start the API
uvicorn app.main:app --reload --port 8000
```

Then:

- API root → `http://localhost:8000/`
- Swagger → `http://localhost:8000/docs`
- Admin → `http://localhost:8000/admin`
- MinIO console → `http://localhost:9001` (`minioadmin` / `minioadmin`)

`config.py` rejects default `SECRET_KEY/ADMIN_PASSWORD`. For throwaway local runs only, set `ALLOW_INSECURE_DEFAULTS=true`.

Storage modes

STORAGE_ENDPOINT	Mode
empty ""	AWS S3 native (uses IAM/task role or STORAGE_ACCESS_KEY/SECRET_KEY)
local	Local filesystem at LOCAL_STORAGE_PATH (./data)
http://localhost:9000	MinIO / custom S3-compatible endpoint

Deployment (AWS)

Region `ap-south-1` (Mumbai).

Container runs on **ECS Fargate Express Mode** (256 CPU / 512 MB, autoscale 1→3 on request count), fronted by an auto-provisioned **ALB** (HTTPS, ACM cert), health-checked at `/health`. Image lives in **ECR** (`ugac-backend`). Secrets come from **Secrets Manager** (prefix `ugac/`) injected as env vars. On container start, `start.sh` runs `alembic upgrade head` then launches uvicorn.

External managed services: **Neon** (Postgres), **Upstash** (Redis), **S3** (`ugac-files`).

CI/CD — GitHub Actions

Workflow: `.github/workflows/deploy-aws.yml`. Triggers on push to `main` touching `backend/**`.

```
graph TD
    A[push to main (backend/**)] --> B[OIDC auth → AWS (role: github-actions-deploy, no static keys)]
    B --> C[docker build → push to ECR (tag = commit SHA)]
    C --> D[render task definition with new image]
    D --> E[deploy to ECS → canary roll (5% / 3 min → full) → wait for stability]
```

One-time setup: add repo secret `AWS_DEPLOY_ROLE_ARN` (the `github-actions-deploy` role ARN) under Settings → Secrets and variables → Actions.

Manual deploy

Replace `<ACCOUNT_ID>`, `<CLUSTER>`, `<SERVICE>` with your real values.

```
ECR=<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/ugac-backend
docker build -t ugac-backend backend/
docker tag ugac-backend:latest $ECR:latest
docker push $ECR:latest
aws ecs update-service --cluster <CLUSTER> --service <SERVICE> --force-
new-deployment
```

Production env vars

Plain: `PORT=8000`, `STORAGE_ENDPOINT=https://s3.ap-south-1.amazonaws.com`,
`STORAGE_BUCKET=ugac-files`, `AWS_REGION=ap-south-1`. (Empty value also routes to S3 native —
both persist to S3.)

From Secrets Manager: `DATABASE_URL`, `REDIS_URL`, `REDIS_TOKEN`, `SECRET_KEY`, `ADMIN_USERNAME`,
`ADMIN_PASSWORD`, `CORS_ORIGINS`, `GITHUB_CLIENT_ID`, `GITHUB_CLIENT_SECRET`,
`STORAGE_ACCESS_KEY`, `STORAGE_SECRET_KEY`.

Logs

```
aws logs tail <CLOUDWATCH_LOG_GROUP> --follow
```

Repo layout

```
UGAC-Web/
├── backend/
│   ├── app/           # FastAPI application (see Layers table)
│   ├── alembic/       # DB migrations
│   ├── docker-compose.yml # local Postgres + MinIO
│   ├── Dockerfile
│   ├── start.sh       # migrate + launch
│   └── requirements.txt
├── .github/workflows/
└── deploy-aws.yml     # CI/CD
```